

---

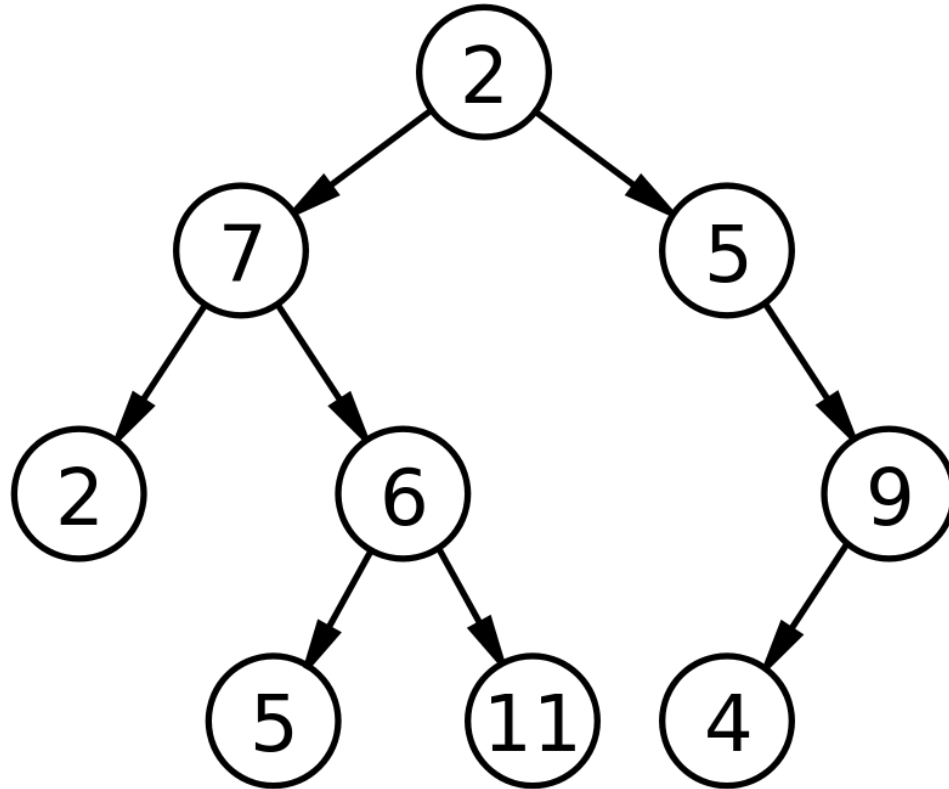
# Data Structures

## Binary Search Tree

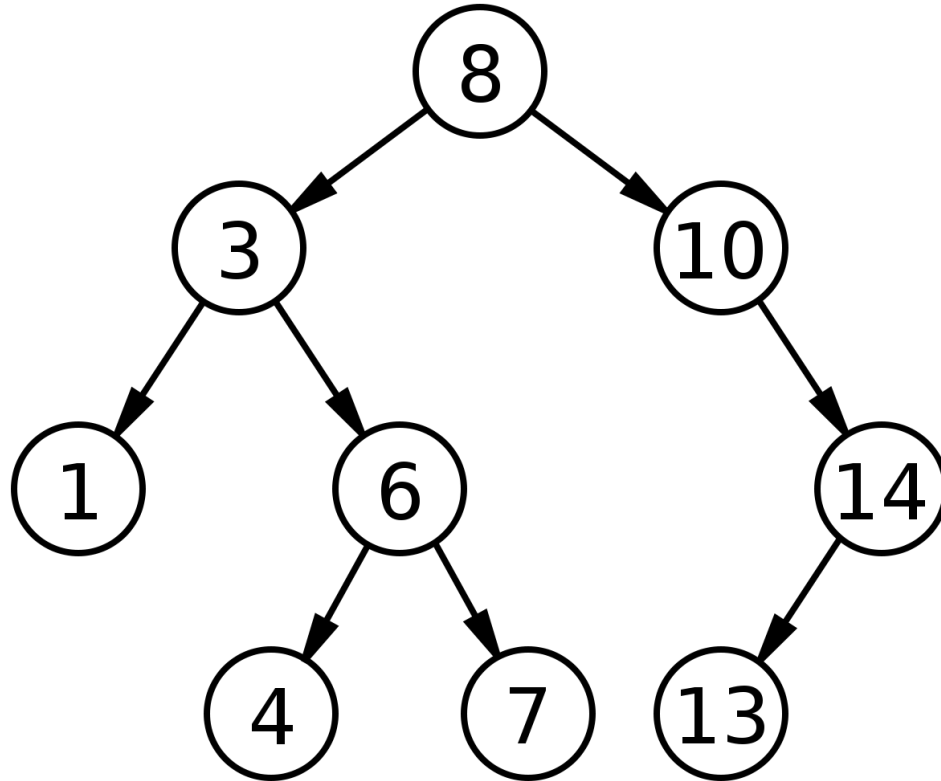
---

Slides taken from Prantik Paul [PKP]

# Binary Tree



## Binary Search Tree



## Binary Search Tree - Restrictions

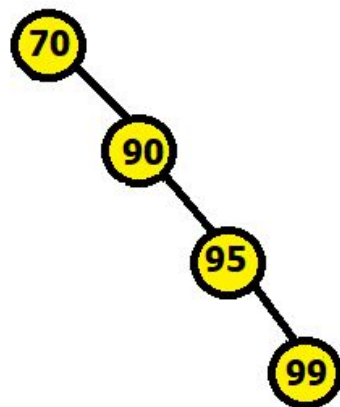
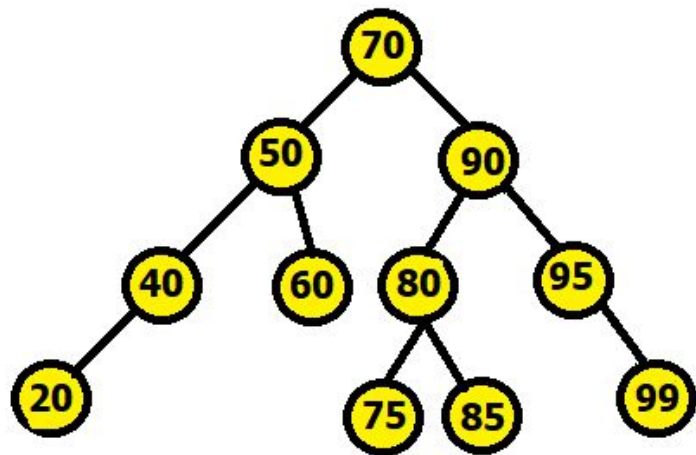
**Element of Left Child  $\leq$  Element of Parent**

**Element of Right Child  $\geq$  Element of Parent**

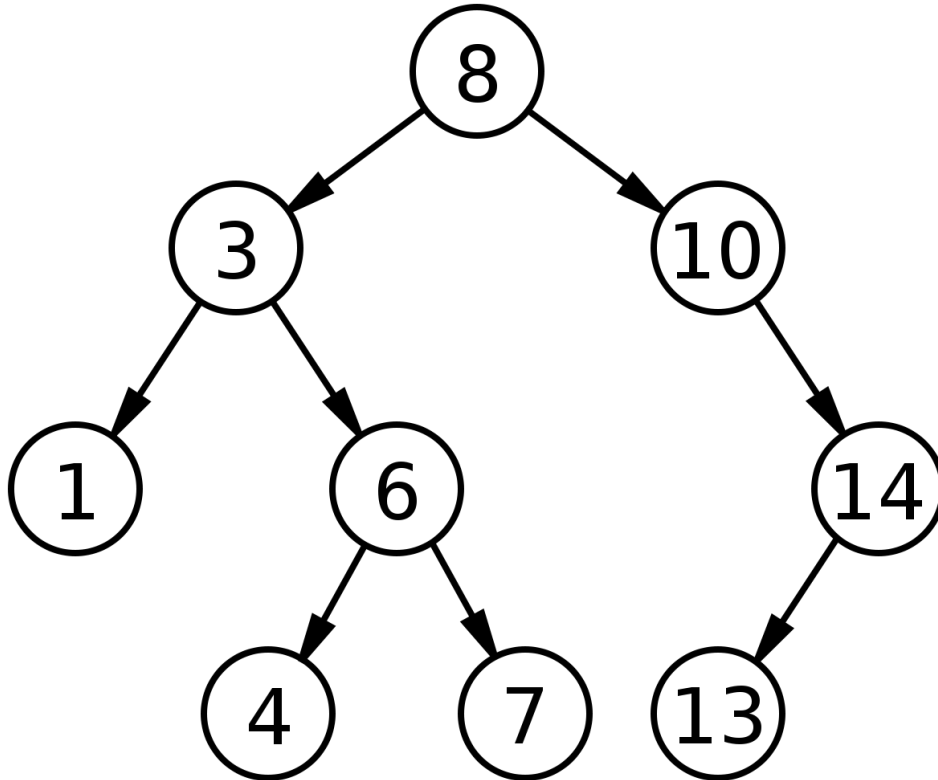
**Left and Right Subtree also BST's**

**No Duplicate Values**

## Binary Search Tree



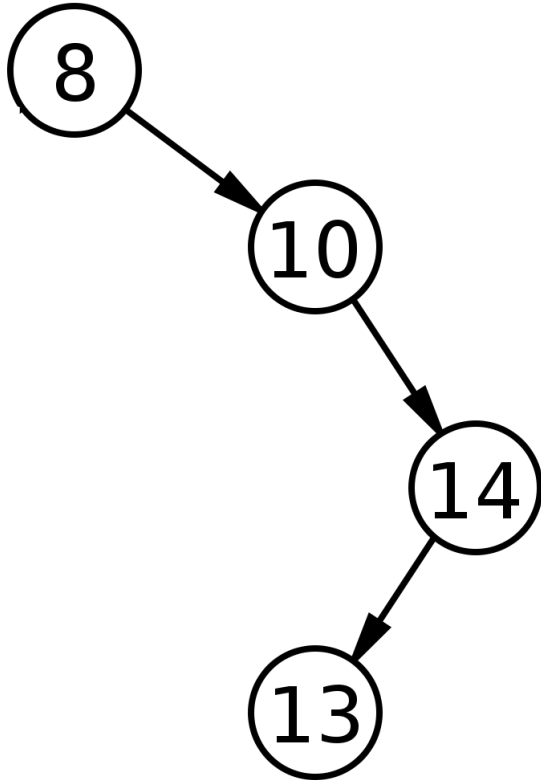
## Binary Search Tree - Why to use



**Search 6**

**Search 5**

## Binary Search Tree - Why to use



**Search 12**

**Must be Balanced**

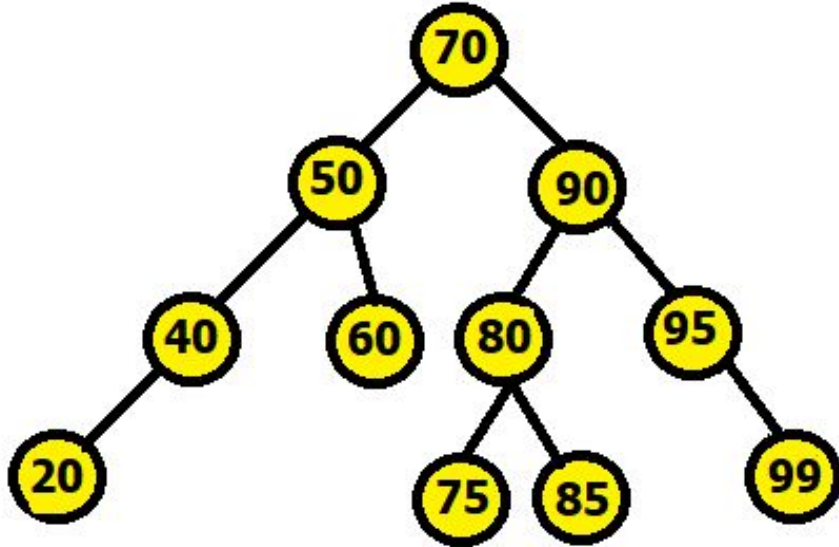
## Binary Search Tree - Creation

**70, 50, 40, 90, 20, 95, 99, 80, 85, 75**

## Binary Search Tree - Creation

**70, 50, 40, 90, 20, 95, 99, 80, 85, 75**

## Binary Search Tree - Insertion



**Insert 10**

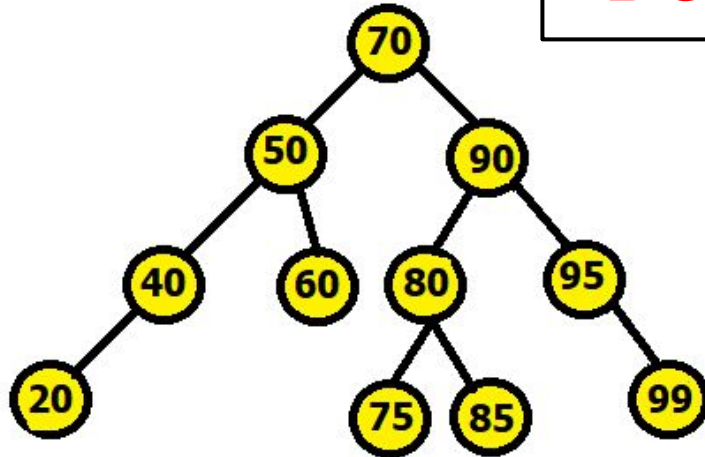
**Insert 5**

**Must Make Balanced**

# Binary Search Tree - Deletion

## Case 1: No Subtree or Children

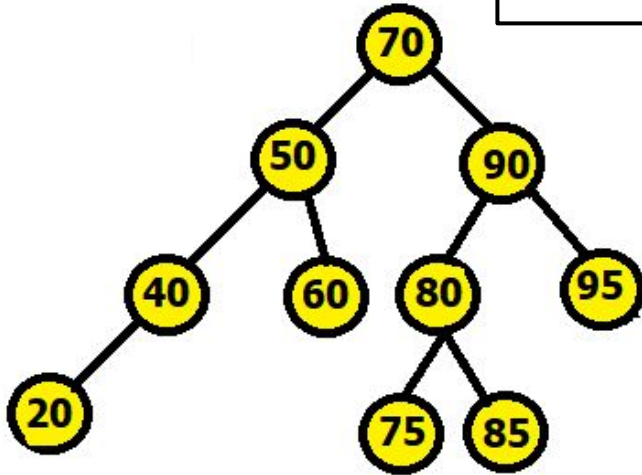
Delete 99



# Binary Search Tree - Deletion

## Case 2: Only 1 Subtree or Child

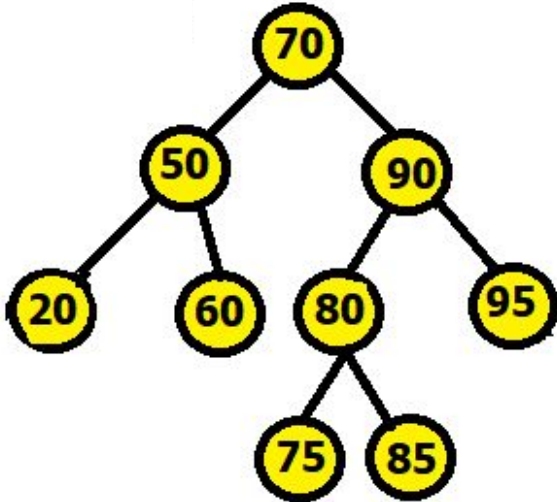
Delete 40



# Binary Search Tree - Deletion

**Case 3: 2 Subtrees or Children** (Replace by **Inorder Successor**)

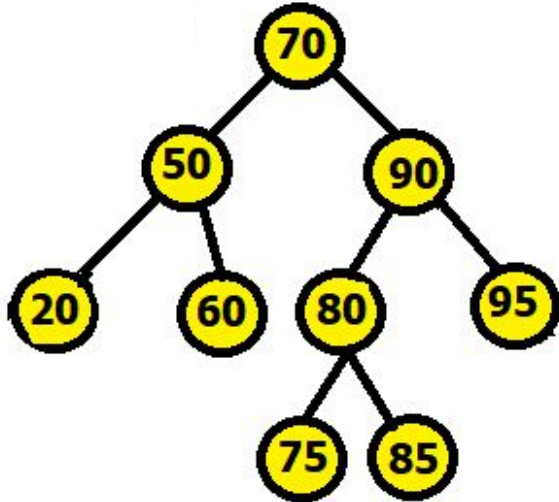
**Delete 90**



# Binary Search Tree - Deletion

**Case 3: 2 Subtrees or Children (Replace by Inorder Predecessor)**

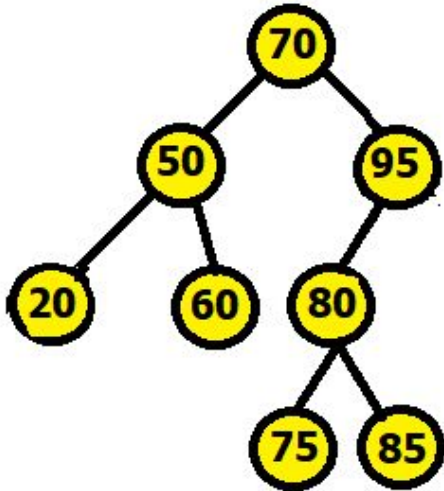
**Delete 90**



# Binary Search Tree - Deletion

**Case 3: 2 Subtrees or Children** (Replace by **Inorder Successor**)

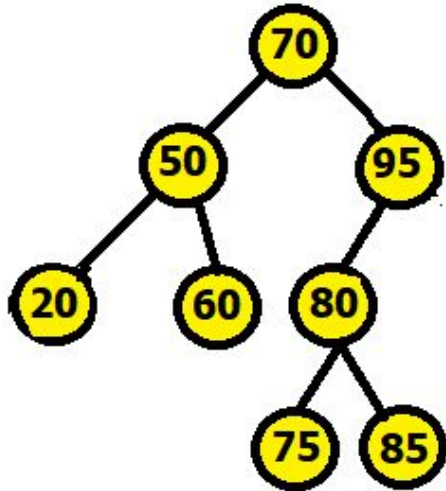
**Delete 70**



# Binary Search Tree - Deletion

**Case 3: 2 Subtrees or Children** (Replace by Inorder Predecessor)

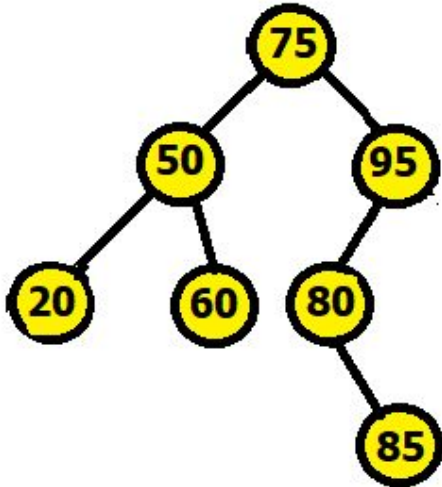
**Delete 70**



# Binary Search Tree - Deletion

**Case 3: 2 Subtrees or Children** (Replace by **Inorder Successor**)

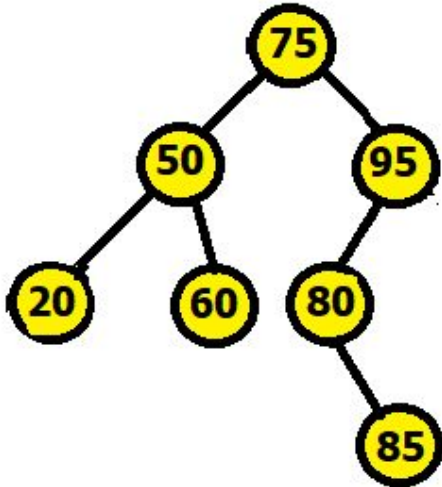
**Delete 75**



# Binary Search Tree - Deletion

**Case 3: 2 Subtrees or Children** (Replace by **Inorder Predecessor**)

**Delete 75**



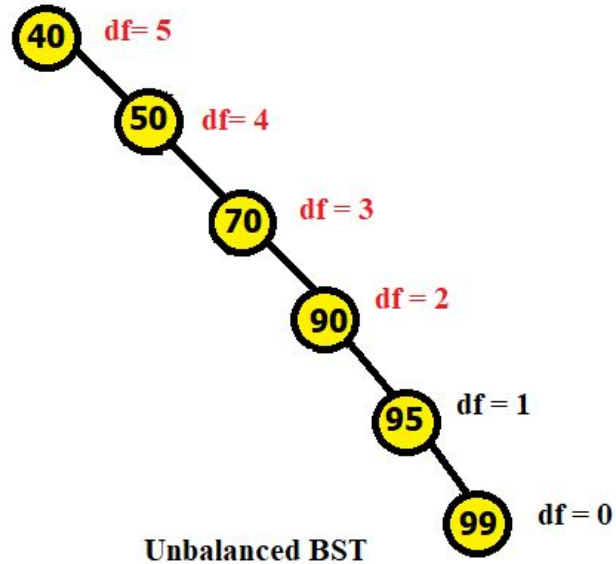
# Binary Search Tree - Balancing

1. Insertion 2. Deletion 3. Traversal 4. Searching 5. Insertion 6. Deletion 7. Traversal 8. Searching 9. Insertion 10. Deletion 11. Traversal 12. Searching

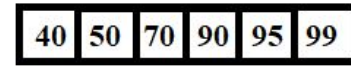
1

1. Insertion 2. Deletion 3. Traversal 4. Searching

# Binary Search Tree - Balancing



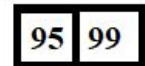
Inorder Traversal



root

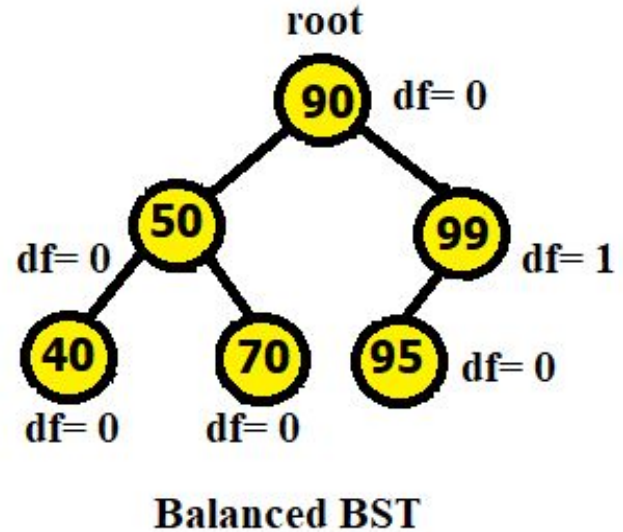
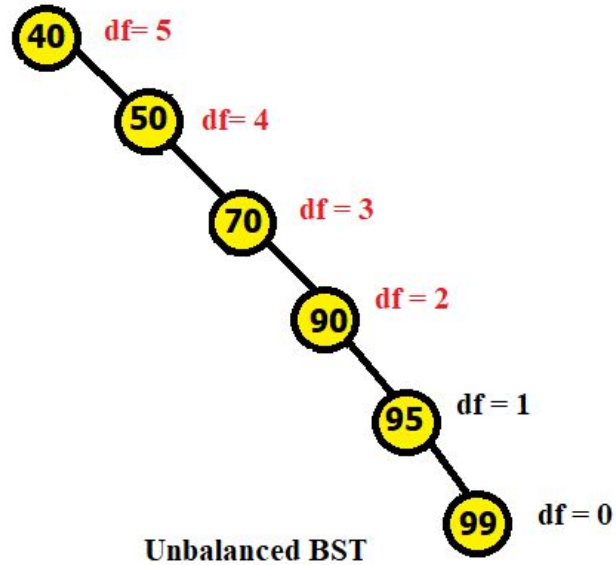


array for  
left subtree



array for  
right subtree

# Binary Search Tree - Balancing



## Binary Search Tree - Insert Node

```
def addNode(root, i): #Adding Nodes
    if i<root.elem and root.left==None:
        n= Node(i)
        root.left= n
    elif i>root.elem and root.right==None:
        n= Node(i)
        root.right= n
    if i<root.elem and root.left!=None:
        addNode(root.left, i)
    elif i>root.elem and root.right!=None:
        addNode(root.right, i)
```

## Binary Search Tree - Find Min

```
def minValueNode(node):  
    current = node  
    while(current.left is not None): # loop down to find the leftmost leaf  
        current = current.left  
    return current
```

# Binary Search Tree - Delete Node

```
10 def deleteNode(root, key):
11     if root is None:
12         return root
13     # If the key to be deleted is smaller than the root's key then it lies in left subtree
14     if key < root.elem:
15         root.left = deleteNode(root.left, key)
16     # If the kye to be delete is greater than the root's key then it lies in right subtree
17     elif(key > root.elem):
18         root.right = deleteNode(root.right, key)
19
20     # If key is same as root's key, then this is the node to be deleted
21     else:
22         # Node with only one child or no child
23         if root.left is None:
24             temp = root.right
25             root = None
26             return temp
27         elif root.right is None:
28             temp = root.left
29             root = None
30             return temp
31         # Node with two children:
32         # Get the inorder successor (smallest in the right subtree)
33         temp = minValueNode(root.right)
34         # Copy the inorder successor's content to this node
35         root.key = temp.elem
36         root.right = deleteNode(root.right, temp.elem) # Delete the inorder successor
37     return root
```

## Binary Search Tree - Balance Tree

```
#Appends into a list with tree nodes using inorder traversal
def pushTreeNodes(root, arr):
    if root is None:
        return
    pushTreeNodes(root.left, arr)
    arr.append(root)
    pushTreeNodes(root.right, arr)
```

## Binary Search Tree - Balance Tree

```
# Recursive function to construct a height-balanced BST from
# given nodes in sorted order
def buildBalancedBST(arr, start, end):
    if start > end:
        return None
    mid = (start + end) // 2 # find the middle index
    root = arr[mid] # The root node will be a node present at the mid-index
    # recursively construct left and right subtree
    root.left = buildBalancedBST(arr, start, mid - 1)
    root.right = buildBalancedBST(arr, mid + 1, end)
    return root
```